

Épique EP-04 - Référentiel RH & Chantres

 Le Master Prompt à copier

Objet : Conception détaillée de l'Épique EP-02 - Référentiel RH & Chantres

Rôle : Agis en tant que Product Owner et Lead Développeur expert en écosystèmes FastAPI / SQLAlchemy.

Contexte : > Nous développons l'API "MLA" pour la gestion du planning d'un ministère de louange. L'Épique 01 (Socle Organisationnel) est terminée. Nous devons maintenant implémenter l'**Épique 02 : Référentiel RH & Chantres**.

Données sources (Modèles SQLAlchemy concernés) :

- **Membre** (lié à Ministère/Pôle)
- **Chantre** (lié à Membre)
- **Choriste** (lié à Chantre et Voix)
- **Musicien** (lié à Chantre et Instrument)
- Tables de référence : **Voix**, **Instrument**

Livrables attendus :

1. **Documentation Confluence (Niveau Stratégique) :**

- **Objectif de l'Épique :** Définir la valeur métier de la centralisation des compétences.
- **Règles Métiers (Business Rules) :** Liste exhaustive (ex: un Chantre peut être à la fois Choriste et Musicien, l'unicité du membre, la gestion des voix par défaut).
- **Diagramme de flux logique :** Expliquer le passage de "Membre" à "Chantre spécialisé".

2. **Backlog JIRA (Niveau Opérationnel) :** Génère des tickets JIRA détaillés.

Pour chaque ticket, inclus :

- **Titre et User Story :** Format "En tant que [Rôle], je veux [Action] afin de [Bénéfice]".
- **Critères d'Acceptation (AC) :** Liste technique des endpoints (ex: **POST** **/chantres**), validations de schémas Pydantic, et codes de retour HTTP

(201, 400, 404).

- **Stratégie de Test (Pytest)** : Détaille les tests unitaires et d'intégration nécessaires (ex: tester qu'un Choriste ne peut pas être créé sans une Voix valide, tester la relation One-to-One entre Utilisateur et Membre).

Contraintes techniques à respecter :

- Utilisation de **UUID v4** pour tous les IDs.
- Gestion rigoureuse des **clés étrangères** (un membre doit appartenir à un `pole_id` et `ministere_id` existants).
- Séparation des schémas Pydantic : `Base` , `Create` , `Update` , et `Read` (pour éviter les récursions infinies dans les relations).

Objectif de l'Épique

Centraliser les informations administratives et les compétences techniques (vocales et instrumentales) de chaque membre du ministère. L'objectif est de permettre au moteur de planning (EP-04) de filtrer intelligemment les ressources selon les besoins d'un service (ex: "Besoin d'un Bassiste qui est aussi Soprano").

Règles Métiers (Business Rules)

- **Identité Unique** : Un individu ne peut avoir qu'un seul profil `Membre` . Un `Membre` peut être lié à un seul `Utilisateur` (Relation 1:1).
 - **Spécialisation Cumulative** : Un `Chantre` peut être simultanément `Choriste` (avec une voix principale/secondaire) et `Musicien` (avec un ou plusieurs instruments).
 - **Intégrité Structurale** : Un membre peut être rattaché à un `Ministere` et un `Pole` existants.
 - **Référentiel Strict** : Les voix et instruments doivent correspondre aux codes définis dans les tables de référence (`t_voix` , `t_instrument`).
 - **Délégation de Responsabilité** : Seul un responsable ayant les permissions adéquates peut modifier le profil technique (Choriste/Musicien) d'un membre.
-

📋 2. Backlog JIRA (Vue Opérationnelle)

Groupe 1 : Données de Référence (Préalable)

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-201	Initialisation Voix & Instruments En tant que système, je veux disposer d'un référentiel de voix et d'instruments.	Endpoints: GET <code>/ref/voix</code> , GET <code>/ref/instruments</code> . Peupler <code>t_voix</code> (SOPRANO, ALTO, etc.) via un script de seed.	Tester que l'insertion d'un code de voix non présent dans l'Enum <code>VoixEnum</code> échoue.

Groupe 2 : Gestion des Membres (Administratif)

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-202	CRUD Membres En tant que Responsable, je veux créer et gérer les membres du ministère.	POST <code>/membres</code> : Validation UUID pour <code>pole_id</code> et <code>ministere_id</code> .	Test d'intégration : Créer un membre et vérifier la 404 si le <code>pole_id</code> est aléatoire. Test Cascade : Supprimer un

		Schéma MembreRea d excluant les mots de passe.	membre doit supprimer son profil Chantre .
MLA-203	Liaison Utilisateur- Membre En tant qu'admin, je veux lier un compte utilisateur à une fiche membre.	Endpoint PATCH /utilisateur urs/{id}/l ink- membre . Vérifier l'unicité (un membre ne peut pas avoir deux utilisateurs).	Tester la contrainte UniqueCons traint sur membre_id dans la table t_utilisat eur .

Groupe 3 : Spécialisation technique (Métier MLA)

Ticket ID	Titre & User Story	Critères d'Acceptation (AC)	Stratégie de Test (Pytest)
MLA-204	Profil Chantre & Choriste En tant que Responsable, je veux définir la voix d'un membre.	POST /chantres (crée l'entrée pivot). POST /choriste S : liaison avec voix_code .	Test Unitaire : Vérifier que secondaire Voix est optionnel. Test de lecture : GET /membres/{ id} doit

			retourner les détails choriste si existants.
MLA-205	Profil Musicien En tant que Responsable, je veux définir les instruments pratiqués par un membre.	POST /musicien s . Validation du instrument _code contre la table t_instrument .	Tester l'ajout de plusieurs instruments pour un même chantre_id .

Contraintes techniques d'implémentation (Lead Dev)

1. **Schémas Circulaires** : Utilise des "Forward References" (**"Chantre"**) dans tes fichiers de schémas Pydantic pour que **MembreRead** puisse inclure **ChantreRead** sans erreur d'importation.
2. **DTO (Data Transfer Objects)** : * **MembreCreate** : Ne demande que les IDs et infos de base.
 - **MembreRead** : Utilise **lazy="joined"** dans SQLAlchemy pour récupérer le profil chantre en une seule requête SQL.
3. **Sécurité** : Tous les **POST/PATCH/DELETE** de cette épique doivent être protégés par une dépendance vérifiant la permission **SCOPE_RH_MANAGE** .